

Multi-Platform Social Campaign Publisher

Turn one published blog post into a complete social campaign: the right image variant and caption per platform, published through a clean adapter layer — scheduled, idempotent, rate-limit-aware, and status-tracked. All against a provided fake platform server.

Difficulty: Medium–Hard

Self-paced · no deadlines

JavaScript or Python

Public GitHub repo

\$0 · no credit card, ever

THE FLAVOR

One post in → a scheduled multi-platform campaign out; reliability engineering in disguise

YOU WILL MASTER

Adapter architecture · idempotency · rate limits · durable scheduling · signed webhooks

YOUR \$0 STACK

Node or Python · provided fake-platform server · sharp/Pillow · Docker Redis or APScheduler

How to read this document: § 1–2 tell you what this capstone is and whether it's the right pick for you. § 3–9 are the build: rules, features, architecture, the definition of done, and the build phases. § 10–12 are the practical frame: the free tools, the GitHub rules, and exactly how you'll be evaluated. § 13–14 prepare your demo and give you hand-picked, free resources for every phase. You don't need to memorize it — work through the § 8 phases at your own pace and come back when you need detail.

Contents

1. The mission.....	1	9. Stretch goals	7
2. What it takes to finish.....	2	10. Your \$0 stack	7
3. Ground rules	2	11. GitHub rules.....	8
4. What you'll build.....	3	12. How it's evaluated	9
5. Architecture overview	4	13. The final demo	10
6. Definition of done	5	14. Curated resources.....	11
7. Realistic scope	5	15. Glossary	12
8. The build, phase by phase	6		

1 · The mission

Take a single published blog post and turn it into a complete social media campaign: generate the correct **image variant** for each platform, write a **platform-tailored caption**, schedule the posts, and publish them through a clean, reliable publishing system that tracks every post from creation to delivery.

Here's the twist that makes this a backend capstone and not a marketing exercise: **the challenge is not calling an API** — it is building a publishing system that survives the real world. Duplicate requests. Network failures. Rate limits. Crashed workers mid-batch. A retry must **never** create a duplicate post. A scheduled campaign must survive a restart. A status may only flip to "published" when a **signature-verified webhook** says so.

You'll build it against a **provided fake social platform server** that faithfully simulates OAuth, access tokens, rate limits ([429](#) + [Retry-After](#)), idempotency keys, delivery webhooks, and random publishing failures. No real Instagram, X, or LinkedIn account is ever touched — the fake server *is* the realism.

FlyRank has the raw materials — platform prompt fragments, an image pipeline, a social status data shape — but **no actual publishing system**. Storage exists; publishing doesn't. You build the part that doesn't exist yet.

Why this one is satisfying: the demo is theatrical. You'll schedule a post for "later", advance time, watch the worker publish it; hammer the publish button five times and prove one single post exists; forge a webhook and watch it bounce. Reliability made visible.

2 · What it takes to finish

Honest picture before you commit — **Medium–Hard**, and here is where the difficulty actually lives:

The three genuinely hard parts

- **Idempotency under retries.** "Same request twice = one post" sounds simple until a timeout happens *after* the platform accepted the post but *before* you heard back. Getting the idempotency-key flow right is the heart of the grade.
- **A durable scheduler.** A worker that publishes at 9:00, crashes mid-batch, restarts, and continues *without double-posting* forces you to think about state, not just code.
- **Trust boundaries.** Status only changes on a signature-verified webhook; forgeries get [400](#) . Tokens are encrypted at rest with a random IV. Security as architecture, not an afterthought.

Time budget: roughly **40–55 focused hours**, at your own pace. The image/caption pipeline is fun and quick; reserve your deep-work blocks for Phase 3 (adapters + idempotency) and Phase 4 (scheduler + webhooks).

You already have the parts

Piece of this capstone	Where you already built it
Validation + idempotent endpoints	A5 / A6
Caption generation with prompt fragments	A11 + Challenge 3
Scheduled background jobs	A9
Adapter + signed webhook patterns	A14

Pick this one if you want the most visibly product-like result and a taste of real distributed-systems thinking. It pairs beautifully with a portfolio: everyone understands what "post to social media" means.

3 · Ground rules — read before you start

These rules are the same for every capstone in the track. They exist so that 20,000+ interns can be evaluated fairly — and so your finished project is something you can safely show in an interview.

The five rules

Rule	What it means for you
Pick one, early	This internship is self-paced — no deadlines. Still, choose your capstone early (once the first assignments have shown you the landscape) and write a one-page design doc (problem, data model, API surface, layer sketch, one explicit non-goal). Phase 1 in § 8 is exactly that doc.
One separate, public repo	The capstone lives in its own public GitHub repository from day one — never inside your assignments repo. Full rules in § 11.
\$0, no credit card — ever	Everything can be built with free tools; this document lists the exact free stack in § 10. If you ever find yourself on a page asking for a credit card, stop — you took a wrong turn, the free path exists.
AI-assisted building is encouraged — and owned	Use AI tools freely, but keep BUILDLLOG.md honest: where AI helped, where it was wrong, what you changed. At the demo you will be asked to explain 2–3 lines the evaluator picks. <i>"The AI wrote it"</i> is not an answer.
Propose your own	You may pitch a different capstone idea against the shared requirements (§ 12). If it exercises a comparable skill set, mentors can approve it.

SANDBOX-FIRST — READ BEFORE YOU BUILD

- The core project must use the provided fake social platform server (`starters/challenge-5-social/`). Do **not** connect to real social media accounts. At 20,000-intern scale, live platform APIs mean account bans, leaked credentials, and terms-of-service violations.
- The fake server simulates everything that matters: OAuth, access tokens, rate limits (`429 + Retry-After`), idempotency keys, delivery webhooks, and publishing failures. **The goal is a realistic production system without touching real accounts.**
- Real platform publishing is an **opt-in stretch goal only** — personal developer app, never shared credentials, never a committed token.
- Tokens are stored **encrypted at rest** (random IV, mirror `lib/serverUtils.ts`) and never logged.

4 · What you'll build

Given a published post (title + body + URL), you build a service with five parts:

1 Image variant pipeline

teaches: media pipelines

From one source image, generate platform-correct variants:

```
Original image
├─ Instagram version  1080×1080  (1:1 square)
└─ X version          1600×900   (16:9 landscape)
```

Requirements: correct dimensions and aspect ratio per platform, the main subject kept inside the **safe zone**, optional branding overlay. The source can be AI-generated or a placeholder — **the graded part is the variant pipeline, not artistic quality.**

2 Platform-tailored caption generation

teaches: composable prompts

Different platforms, different voices — composed from **reusable prompt fragments**, not copy-pasted prompts:

```
Shared brand voice + Platform rules + Content summary
→ Instagram caption · X caption · LinkedIn caption
```

Model it on FlyRank's real `config/social-prompts.config.ts`. No near-identical duplicated prompts.

3 Social publishing adapter layer

teaches: adapter architecture

One `SocialPublisher` interface, at least two platform adapters against the fake server:

```
SocialPublisher
├─ FakeInstagramPublisher
└─ FakeXPublisher
```

Each adapter handles OAuth tokens (encrypted storage), **idempotent** publishing, rate-limit handling, and retry logic. **Your application should not know which platform it is publishing to** — adding a platform means adding an adapter, not rewriting the app.

4 Durable scheduling system

teaches: durable background work

"Publish this campaign tomorrow at 9:00." A scheduled post is picked up by a **durable worker** at the right time and published through the adapter. The system must survive server crashes, worker restarts, and partial batch failures — a restarted worker continues safely **without publishing duplicates**.

5 Webhook-based status tracking

teaches: webhook trust

The fake platform sends delivery events to `POST /webhook/social-delivery`. Your system verifies the **webhook signature**, rejects forged or modified events with `400`, and only then updates the campaign status:

```
queued → publishing → published | failed
```

Stored using the provided `SocialPostEntry` model shape. Status is trust, and trust requires verification.

5 · Architecture overview

Content flows down the left, reliability machinery guards the right — every publish passes through the same interface, no matter the platform:

```
Blog Post
├─ Caption Composer (shared + platform fragments) → per-platform captions
└─ Image Variant Pipeline → per-platform images (sized, safe-zone)

Campaign created → scheduled for 09:00
├─ Durable Queue -(worker wakes at time)→ SocialPublisher interface
│   └─ FakeInstagram adapter
│   └─ FakeX adapter
└─ guards on every publish: idempotency key · 429/Retry-After backoff · encrypted token
```

```
└─ FAKE SOCIAL PLATFORM → signed delivery webhook → verify signature
                                     └─ valid → status: queued →
published | failed
                                     └─ forged → 400 rejected
```

6 · Definition of done — the core checklist

This is the contract. **Done = every box below ticked, with one pasted proof per box in `EVIDENCE.md`**. Each box is written so a reviewer can verify it in minutes.

CONTENT GENERATION

- ☐ Platform image variants generated correctly: dimensions, aspect ratio, subject inside the safe zone — a test asserts the dimensions.
- ☐ Captions are platform-aware and composed from shared + platform-specific fragments; no duplicated near-identical prompts.

ADAPTER LAYER

- ☐ One `SocialPublisher` interface with at least two implementations; the application depends on the interface, never a platform.
- ☐ OAuth tokens encrypted at rest with random-IV encryption; never logged, never committed.

RELIABILITY

- ☐ Idempotent publishing: the same (post, platform) published twice — or retried after a timeout — yields exactly **one** post.
- ☐ Rate limits respected: on `429`, the worker honors `Retry-After`, backs off, and retries safely without hammering the API.
- ☐ Scheduling is durable: a scheduled post publishes on time; a crash mid-batch resumes without double-posting.

STATUS & TRUST

- ☐ Delivery webhooks are signature-verified; forged or modified webhooks are rejected with `400`.
- ☐ Campaign status transitions (`queued → published | failed`) update only after a verified webhook, stored in a `SocialPostEntry`-shaped record.

TESTS & DOCUMENTATION

- ☐ Automated tests cover: image dimensions per platform, duplicate-publish prevention, forged-webhook rejection, and rate-limit behavior.
- ☐ README + architecture diagram + setup instructions; everything runs against the fake platform server; submission-pack files from § 11 present.

7 · Realistic scope — where to stop

Keep the core focused — reliability is the goal, not quantity:

- **Two platforms are enough** (e.g. "instagram" 1:1 + "x" 16:9). Do not spend time building many integrations.
- **Image generation can be a resize/crop of a placeholder**. Variant *spec correctness* is graded, not artistry. Generating the source with a free AI model is encouraged but optional.
- **The campaign view can be tiny** — a validated API plus a simple status table.
- **The whole publish path runs against the fake platform**. Never a real API in the core build.

8 · The build, phase by phase

This internship is **self-paced** — there is no calendar and no deadline. Work through the phases **in order, at your own speed**: the track assignments are the parts, the capstone assembles them. Each phase ends with a **gate** — a concrete result that tells you it's safe to move on. The effort estimates are just orientation; take what you need. Short on time overall? Shrink scope (§ 7), don't skip phases.

Phase 1 Design · ≈4–6 h

gate: design doc signed off

- Platform specifications: sizes, aspect ratios, voice rules
- The `SocialPublisher` interface signature
- Data model: campaigns, posts, statuses, tokens

Phase 2 Content generation · ≈8–12 h

gate: real image files + distinct captions produced

- Image variant pipeline producing correctly-sized artifacts
- Caption composition from shared + platform fragments

Phase 3 Publishing system · ≈14–18 h

gate: duplicate publish = one post; 429 respected

- Adapter layer with ≥ 2 fake-platform adapters
- Idempotent publish with idempotency keys
- Rate-limit handling (429 + `Retry-After` + backoff)
- Encrypted token storage

Phase 4 Production reliability · ≈12–16 h

gate: forged webhook → 400; crash-resume without dupes; tests green

- Durable scheduler + worker with crash recovery
- Signature-verified delivery webhook → status tracking
- Tests + README + `EVIDENCE.md` filled as you go

Phase 5 Demo prep · ≈2–3 h

gate: § 13 rehearsed twice, failure moments prepared

- Seed a campaign, rehearse time-advance, the idempotency hammer, and the forged webhook

9 · Stretch goals — only if the core ships

★ **Optional, and only after every § 6 box is green.** A finished core with one polished stretch beats three half-stretches. Each of these is a genuine "I went deep" interview story:

- **Real platform publishing** — opt-in and careful: your own personal developer app, tokens never committed.
- **Brand templating**: logo overlays and per-store branding configuration.
- **A/B captions**: stable-hash variant selection (reuse the Challenge 3 stretch).
- **Approval workflow**: `draft` → `approved` → `scheduled` → `published`.
- **Analytics loopback**: ingest likes/clicks from the fake platform.
- **Run it as a node** on the Streams platform (Capstone 1 family).

10 · Your \$0 stack — the free-tools promise

We promised you can finish this internship without paying for anything. Here is the proof for this capstone. Every requirement maps to a tool that is free with **no credit card**:

You need	Free tool (no credit card)	Notes
Language + framework	Node.js + Express or Python + FastAPI	Free, as all track long
The social platforms	Provided fake-platform server (<code>starters/challenge-5-social/</code>)	Free · runs locally · no real accounts ever
Database	PostgreSQL via Docker (or SQLite)	Free
Image processing	sharp · Pillow	Free libraries
Captions (optional AI)	Gemini free tier or local Ollama — or hand-written templates	Free · AI optional, composition is what's graded
Scheduling / queue	BullMQ + Redis (Docker) · APScheduler with a persistent job store	Free
Encryption	Node crypto (AES-GCM) · cryptography (AESGCM)	Built-in / free
Source images	Placeholder images, or any free AI image tool	Free · artistry is not graded

The iron rule: if any tool, tier, or tutorial asks for a credit card, it is the wrong path — a free alternative for this capstone exists in the table above. Stuck anyway? Ask in the community before paying for *anything*.

11 · GitHub rules — your public repo

Your capstone is also your **portfolio piece**. These rules make the repo something a recruiter, a mentor, and our automated evaluator can all navigate without asking you anything.

The non-negotiables

- ☐ **One dedicated repository, public from day one.** Create it the day you choose your capstone. Do **not** build in your assignments repo, a private repo you flip later, or a monorepo shared with other work. Progress in the open is part of the story — an honest half-built history beats a single "final version" push.
- ☐ **Name it clearly:** `flyrank-capstone-<short-name>` (for this capstone we suggest `flyrank-capstone-social-studio`). Lowercase, hyphens, no spaces.
- ☐ **Commit as you build.** Small, meaningful commits with messages that say what changed (`Add idempotency key check to publish endpoint`, not `update stuff`). Aim for at least one commit per working session; each phase in § 8 should be visible in your history.
- ☐ **Never commit a secret.** No API keys, tokens, passwords, or `.env` files — put `.env` in `.gitignore` before your first commit and ship a `.env.example` with placeholder values instead. A leaked key in your history means rotating the key and rewriting history — ask for help the moment it happens.
- ☐ **A stranger can run it.** The README's setup section must work on a clean machine with one documented run command (`docker compose up` or equivalent) plus a seed step for demo data.

Required files at submission

File	What goes in it
<code>README.md</code>	What the system does, an architecture diagram (an image or ASCII sketch), exact run + seed steps, and an honest "limitations" note.
<code>capstone.yaml</code>	A small manifest the evaluator reads: <code>run:</code> (one command), <code>seed:</code> , <code>test:</code> , <code>base_url:</code> and the endpoints to probe.
<code>EVIDENCE.md</code>	One pasted proof per Definition-of-Done checkbox in § 6 — a test name + output, a curl transcript, or a log line. Claims without evidence score as not done.
<code>BUILDLOG.md</code>	Your AI-usage log: where AI helped, where it was wrong, what you changed. Honesty is graded, perfection is not.
<code>.env.example</code>	Every environment variable the app needs, with safe placeholder values.

DO

- Create the repo public before you build, first commit = README skeleton + `.gitignore`.
- Add a license (MIT is a fine default) so others may read and learn from it.
- Use branches if you like — but keep `main` always runnable.
- Paste real command output into `EVIDENCE.md` as you finish each checkbox — not in a panic at the end.

DON'T

- Don't mix capstone code with track assignments — separate repos, always.
- Don't commit `node_modules/`, `virtualenvs`, or datasets over a few MB.
- Don't force-push over your history before submission — the journey is evidence.
- Don't commit real tokens "just for a second". There is no such thing on a public repo.

12 · How your capstone is evaluated

Three layers, published up front — **you know exactly what will be checked, so build to pass it.**

Layer 1 — The submission pack (machine-checkable)

The portal first checks your repo structure: the five required files from § 11, a `run:` command that boots the system, and a `test:` command that runs green. Missing pack files cost points before a human ever looks.

Layer 2 — Acceptance probes (behavioral, pass/fail)

An evaluator (human or automated) runs these against your live system. They are not secrets — they are promises:

PROBE 1 — Publish the same campaign twice, then retry after a simulated timeout → exactly **1** published post per platform, not 3.

PROBE 2 — The fake platform returns `429 Retry-After: 30` → your worker waits, retries once allowed, and the publish succeeds without hammering.

PROBE 3 — Schedule a post, kill the worker mid-batch, restart it → publishing completes with zero duplicates.

PROBE 4 — Send a forged delivery webhook (bad signature) → `400`, status unchanged. Send the valid one → status flips to published.

PROBE 5 — Inspect generated artifacts → Instagram file is 1080×1080, X file is 1600×900, captions differ per platform.

PROBE 6 — Grep the database and logs → no plaintext token anywhere; stored tokens are encrypted.

Layer 3 — The rubric (judgment, 1–5 per dimension)

The weights say what we value: **correctness and resilience over feature count.**

Dimension	What a "5" looks like	Weight
Architecture	Clean layers; swap the DB or a provider without touching business logic	×3
Correctness	Happy path solid; edge cases handled; probes all pass	×3
Resilience	Survives bad input, a dead dependency, a job running twice	×3
Security	Real authorization; validated input; secrets encrypted & never logged	×2
AI cost & grounding (if AI used)	Cost metered + capped; outputs grounded; eval catches regressions	×2
Testing	The scary cases, deterministically; evals for AI behavior	×2
Communication	README + diagram tell the story; honest about limits; demo lands	×2

Levels: **Ships** = every core box in § 6 done + all probes pass (a real pass) · **Solid** = Ships + rubric average ≥ 3.5 · **Exceptional** = Solid + meaningful stretch goals. A small system that is correct, resilient and well-tested beats a huge one that falls over — that is what senior engineers actually value.

The shared requirements (every capstone must show these)

These eight patterns are how we know you absorbed the program, not just this topic. You built each one in an assignment already:

#	Requirement	Where you learned it
1	Layered architecture — data / logic / HTTP separated	A2 + the architecture live event
2	Validation at the boundary — bad input → clean 4xx, never a 500	A5
3	≥1 background job — slow/bulk work off the request path, retries + failure alert	A7
4	Real persistence — schema as migrations, right indexes, isolated tenants	A5
5	Idempotency where it matters — the retried action happens once	A5 stretch + A10
6	Secrets clean — env only, encrypted if stored, never logged	A10
7	Cost tracked, if AI is used — per call, attributed, with a budget guard	A6
8	Tests that matter — the scary cases, deterministic; AI features get an eval	A13

13 · The final demo — your 6 minutes

Rehearse this flow start to finish. A demo that shows **one failure handled gracefully** impresses more than ten happy paths.

1. Select a blog post and hit **"Create Campaign."**
2. Show the generated images — square for Instagram, wide for X — and the different captions (point out how the platform voices differ).
3. Schedule a post for "later". Advance time. Watch the worker publish it automatically.
4. Hammer publish repeatedly and prove **one request = one post** — the idempotency moment.
5. Trigger a fake rate limit (`429 Retry-After`) — show your backoff handling it calmly.
6. Send a forged delivery webhook → **rejected**. Send the valid one → status flips to published.
7. Close on the campaign dashboard, green across platforms — **and not one real social account was ever touched**.

Study these FlyRank parts first

FlyRank reference	What to steal from it
<code>config/social-prompts.config.ts</code>	Platform voice as data: caption fragments, brand style — the model for your composer
<code>SocialPostEntry</code> / <code>ContentSocials</code> in <code>types/content.types.ts</code>	The status shape to model your records on. Storage exists; publishing doesn't — that's your job
<code>inngest/generateImageForContent.ts</code> · <code>lib/dynamic-image-variants/</code>	Image generation + variants: your starting point for the variant pipeline
<code>app/api/webhooks/stripe/route.ts</code> · <code>lib/serverUtils.ts</code>	Webhook signature verification + encryption utilities — mirror these patterns

14 · Curated resources — free, verified, leveled

Don't read everything. Each row says when to reach for it. Every link was verified while writing this brief; every resource is free with no credit card. If a link ever dies, the title is searchable.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** | Lanes: **JS** · **PY** · **both**

Phase 1 · Design

	Lane	Resource	Format	When to use it
●	both	Refactoring.Guru — Adapter pattern	Article, ~15 min	Before designing SocialPublisher — with TS and Python examples.
●	both	Stripe — Designing APIs with idempotency	Article, ~15 min	To understand exactly-once behavior before you spec the publish endpoint.
●	both	AWS — Timeouts, retries, and backoff with jitter	Article, ~20 min	When planning your retry policy — why "retries are selfish".
●	both	Sprout Social — Social image sizes guide	Cheat sheet, ~10 min	Why 1080×1080 and 1600×900 matter and how platforms crop.

Phase 2 · Content pipeline

	Lane	Resource	Format	When to use it
●	JS	sharp — resize API	Docs, ~20 min	While building variants: cover / contain fits, gravity for safe-zone crops.
●	PY	Pillow — Handbook tutorial	Docs, ~30 min	While building variants: open, resize, crop, paste with runnable snippets.

Phase 3 · Adapters & publishing

	Lane	Resource	Format	When to use it
●	both	MDN — 429 Too Many Requests	Reference, ~5 min	When implementing rate-limit handling against the fake server.
●	both	MDN — Retry-After header	Reference, ~5 min	To parse both forms (seconds and HTTP-date) correctly.
●	JS	Node.js — crypto module	Docs, ~20 min	For encrypted tokens + HMAC: createCipheriv (AES-GCM), randomBytes , createHmac .
●	PY	pyca/cryptography — AESGCM	Docs, ~10 min	For encrypted tokens: encrypt/decrypt, nonce guidance, why nonce reuse is fatal.

Phase 4 · Scheduling & webhooks

	Lane	Resource	Format	When to use it
●	both	webhooks.fyi — HMAC signatures	Article, ~10 min	Before implementing webhook verification: HMAC, timestamps, replay protection.

● JS	BullMQ — Delayed jobs	Docs, ~5 min	For scheduled publishing: the <code>delay</code> option, rescheduling.
● PY	APScheduler — User guide	Docs, ~25 min	For scheduled publishing: <code>date</code> triggers + persistent job stores so jobs survive crashes.
● JS	Better Stack — Job scheduling with BullMQ	Build-along, ~2 h	Deep build-along for the durable worker: queues, workers, concurrency, monitoring.

15 · Glossary

Plain-language definitions of the **bold** terms in this brief. No definition depends on another — read in any order.

Term	What it means
Adapter (pattern)	A class that translates between your app's interface (<code>SocialPublisher</code>) and one platform's API. Add a platform = add an adapter, touch nothing else.
Idempotency	Doing something twice has the same effect as doing it once. Here: retrying a publish never creates a second post.
Idempotency key	A unique value sent with a request so the server can recognize "I already did this one" and return the original result instead of repeating the work.
Rate limit	An API's cap on how many requests you may send. Exceeding it returns <code>429</code> with a <code>Retry-After</code> header telling you how long to wait.
Backoff	Waiting progressively longer between retries instead of hammering a struggling API.
Durable scheduling	Scheduled work stored somewhere that survives a crash, so a restarted worker picks up where things left off.
Webhook	An HTTP request a platform sends <i>to you</i> when something happens — here, when a post is actually delivered.
Webhook signature	A cryptographic stamp (HMAC) proving a webhook really came from the platform and wasn't modified. Unverified events must be rejected.
OAuth token	The credential proving you may publish to an account. Stored encrypted, never logged, never committed.
Encryption at rest / IV	Encrypting data before storing it. The IV (initialization vector) is a random value making each encryption unique — reusing it breaks the security.
Safe zone	The central region of an image guaranteed visible after each platform's cropping. Your variants keep the subject inside it.
Prompt fragment	A reusable piece of prompt text (brand voice, platform rules) composed with others instead of copy-pasting whole prompts.
Fake platform server	The provided local server that behaves like a real social API — OAuth, 429s, webhooks, failures — without any real account.

FlyRank Internship · Backend Development Track · Capstone — Multi-Platform Social Campaign Publisher. Everything in this brief can be completed with free tools; no resource linked here requires a credit card. Questions → the capstone channel on the community. Version: July 2026.